Assignment-6

Medina Alish, SE-2538

Repository link: aalishmee/sdp6

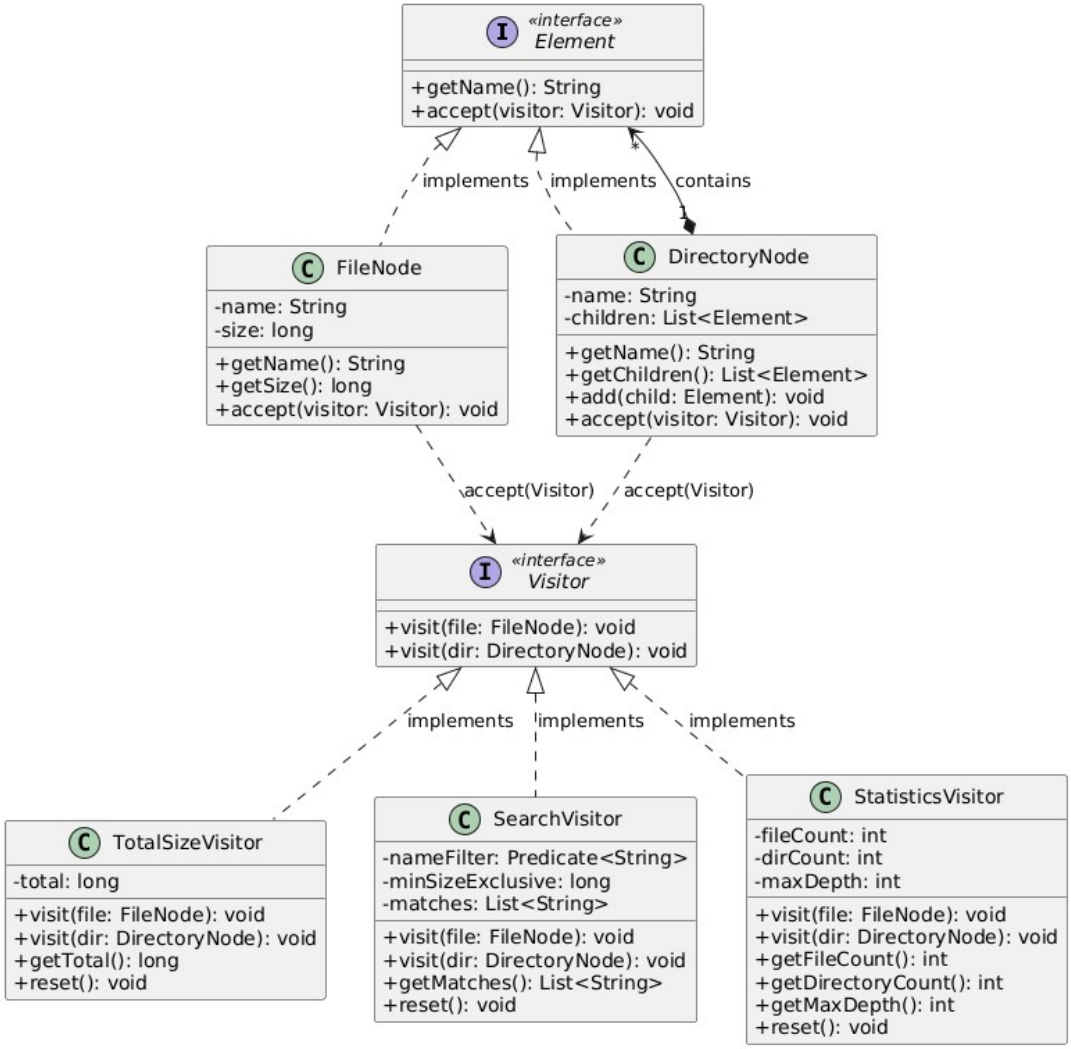
1. Purpose and Design Choices

The goal of this assignment was to build an in-memory hierarchical file system and implement the Visitor behavioral pattern. The system consists of a common Element interface, implemented by two concrete classes: FileNode and DirectoryNode.

To keep the design clean, I made the deliberate choice to let the visitors own the traversal. This means the element classes (FileNode and DirectoryNode) act purely as data structures holding their names, sizes, and children. They do not know how to calculate sizes or search themselves. Instead, they simply accept a visitor and delegate the work. This separation of concerns makes the code highly extensible; if we need a new operation later, we only have to add a new visitor class without modifying the file system structure at all.

Furthermore, to ensure tree integrity (R6), I enforced strict rules in the DirectoryNode.add() method. It rejects duplicate sibling names and prevents nodes from adding themselves. The getChildren() method returns an unmodifiable list, meaning no external class can bypass these rules by directly altering the list of children.

2. UML Diagram



3. Five Clean Code Justifications

Meaningful naming: Methods like getTotal() in TotalSizeVisitor or getMatches() in SearchVisitor clearly describe exactly what they return, making the code read like plain English.

Single Responsibility: FileNode is only responsible for holding a file's data. It does not contain any search or size-calculation logic; that is delegated entirely to the visitors.

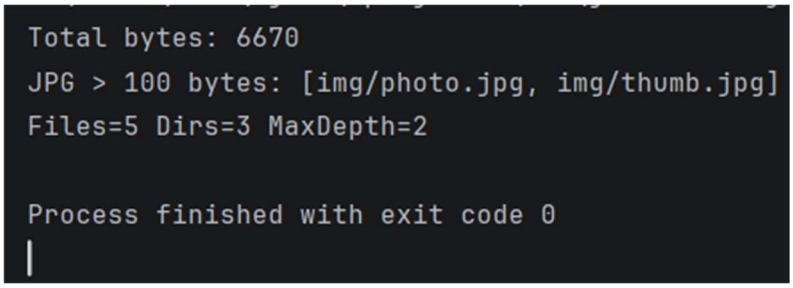
Small methods: Every visit method across the three visitors is only a few lines long, allowing a reader to quickly verify the logic without scrolling.

Encapsulation: DirectoryNode.getChildren() returns Collections.unmodifiableList(children). This prevents external code from corrupting the tree structure.

Elimination of type-checking chains: Instead of using instanceof or getClass() to figure out what type of element we are dealing with, the code relies entirely on double dispatch (accept calling visit), keeping the logic polymorphic and clean.

4. Sample Outputs

Running the Demo.java program produces the following deterministic output:



5. Test Summary

I wrote 10 JUnit 5 tests spread across 5 test classes (DoubleDispatchTest, TotalSizeVisitorTest, SearchVisitorTest, StatisticsVisitorTest, and TreeIntegrityTest). The tests cover checkpoints T1 through T8, including edge cases like empty trees, invalid mutations, and ensuring visitors reset their state properly between runs. Running mvn clean test from the project root results in a BUILD SUCCESS with all 10 tests passing.

6. Traversal and Extension Trade-offs

By giving traversal responsibility to the visitors (R7), the element classes remain completely closed for modification. As required by R8, I added the StatisticsVisitor without changing a single line of code in FileNode, DirectoryNode, or the Visitor interface.

However, the Visitor pattern does have a trade-off: adding a new element type (for example, a SymbolicLinkNode) would require updating the Visitor interface and all three existing visitors to add the new overload. Since adding new operations is far more common in a file system than adding new node types, this trade-off is highly acceptable for this architectur

7. Conclusion

Overall, this assignment was a great exercise in applying the Visitor pattern to a realistic hierarchy. It clearly demonstrated the power of double dispatch and showed how separating data structures from operations leads to much cleaner, more maintainable code. By enforcing strict invariants in the element classes and relying on visitors for all logic, the system successfully achieves a high degree of extensibility. The successful implementation of three distinct visitors and a comprehensive JUnit test suite confirmed that the architecture is both robust and ready for future changes.